Guía de trabajo con Inteligencia Artificial y preparación para la defensa individual

Evaluación Parcial N.º 2 — Taller de Base de Datos (BDY1103)

1. Propósito de esta guía

En esta evaluación podrán utilizar herramientas de Inteligencia Artificial (IA) como apoyo para analizar el problema, diseñar soluciones, desarrollar código PL/SQL, resolver errores, generar pruebas y documentar el proyecto.

El objetivo no es demostrar que pueden producir código sin ayuda, sino que son capaces de utilizar herramientas de IA de manera crítica, comprender las soluciones obtenidas y asumir responsabilidad por su funcionamiento.

Durante el desarrollo deberán preguntarse constantemente:

- ¿Comprendo la solución que estoy construyendo?
- ¿Por qué esta solución es apropiada para el problema?
- ¿Cómo sé que el código funciona correctamente?
- ¿Qué errores o limitaciones encontré?
- ¿Qué decisiones tomé y por qué?
- ¿Qué aprendí gracias al proceso de trabajo con IA?

Esta guía contiene preguntas orientadoras para desarrollar el proyecto, documentar el proceso y prepararse para la defensa individual.

Importante: No es necesario responder todas estas preguntas de manera escrita ni incorporarlas literalmente al informe. Deben utilizarse como orientación para el trabajo y la reflexión. Durante la defensa, el docente podrá formular preguntas de estas categorías u otras relacionadas con la solución desarrollada.

---

2. Comprensión del problema y análisis del negocio

Antes de solicitar código a una herramienta de IA, deben comprender el problema que están intentando resolver.

1. ¿Qué problema de negocio aborda nuestro proyecto?
2. ¿Quiénes se ven afectados por este problema?
3. ¿Qué información necesita generar o procesar el sistema?
4. ¿Qué datos son necesarios para resolver el problema?
5. ¿Qué reglas de negocio deben respetarse?
6. ¿Qué operaciones podrían generar inconsistencias en la base de datos?
7. ¿Qué tareas podrían automatizarse mediante PL/SQL?
8. ¿Por qué una solución implementada directamente en la base de datos podría resultar conveniente?
9. ¿Qué parte del problema no se resuelve simplemente mediante una consulta SQL?
10. ¿Qué requisitos debe cumplir nuestra solución para considerarse correcta?
11. ¿Cómo comprobaríamos que la solución realmente responde a la necesidad del negocio?
12. ¿Qué información necesita conocer la IA para comprender correctamente nuestro caso?
13. ¿Qué ocurriría si le entregáramos a la IA una descripción incompleta o ambigua del problema?
14. ¿Cómo podemos detectar si una propuesta de la IA no responde a nuestras necesidades reales?

3. Planificación del trabajo con IA

La IA puede generar código rápidamente, pero eso no significa que la primera solución sea la más adecuada.

15. ¿Qué herramienta de IA utilizamos y para qué tareas?
16. ¿Qué decidimos solicitarle primero y por qué?
17. ¿Cómo dividimos el problema en tareas más pequeñas?
18. ¿Qué información incluimos en nuestras solicitudes para obtener respuestas pertinentes?
19. ¿Qué diferencia existe entre pedir que la IA genere una solución completa y solicitar que nos ayude a resolver una parte específica?
20. ¿Cómo evitamos que la IA tome decisiones de diseño sin que nosotros las evaluemos?
21. ¿Qué solicitudes tuvimos que reformular para obtener mejores respuestas?
22. ¿Qué información adicional tuvimos que proporcionar durante el proceso?
23. ¿Cómo verificamos que la respuesta de la IA fuera coherente con nuestro modelo de datos?
24. ¿Qué hicimos cuando obtuvimos respuestas contradictorias o poco claras?

25. ¿Qué tareas resolvimos con apoyo de la IA y cuáles requirieron mayor análisis de nuestra parte?
26. ¿En qué momento decidimos no aceptar una sugerencia de la IA?
27. ¿Cómo organizamos el trabajo del equipo para evitar que cada integrante construyera componentes incompatibles?
28. ¿Qué decisiones tomamos antes de escribir código y cuáles surgieron durante la experimentación?

4. Comprensión y validación del código generado

El código generado por IA debe revisarse antes de considerarse parte de una solución válida.

29. ¿Podemos explicar cada sección importante del código que utilizamos?
30. ¿Qué hace cada procedimiento, función, package y trigger de nuestro proyecto?
31. ¿Qué entradas recibe cada componente y qué resultados produce?
32. ¿Qué tablas consulta o modifica?
33. ¿Qué supuestos realiza el código?
34. ¿Qué errores podrían producirse al ejecutarlo?
35. ¿Cómo verificamos que el código compila correctamente?
36. ¿Cómo comprobamos que, además de compilar, produce los resultados esperados?
37. ¿Qué datos utilizamos para realizar nuestras pruebas?
38. ¿Cómo verificamos que los datos se modificaron de la manera correcta?
39. ¿Qué diferencia existe entre obtener un resultado aparentemente correcto y demostrar que la solución funciona?
40. ¿Qué evidencia podemos mostrar para respaldar nuestras conclusiones?
41. ¿Qué parte del código revisamos con mayor atención y por qué?
42. ¿Qué recomendaciones de la IA aceptamos sin modificaciones y cuáles adaptamos?
43. ¿Qué problemas podrían pasar inadvertidos si únicamente copiáramos y ejecutáramos el código generado?
44. ¿Cómo comprobaríamos si una solución funciona con datos distintos de los utilizados inicialmente?
45. ¿Qué haríamos si el código compila, pero los resultados no coinciden con los requisitos del negocio?

5. Procedimientos almacenados

46. ¿Qué problema específico resuelve cada procedimiento de nuestro proyecto?

47. ¿Por qué decidimos implementar un procedimiento en lugar de ejecutar directamente una sentencia SQL?
48. ¿Qué diferencia existe entre un procedimiento con parámetros y uno sin parámetros?
49. ¿Qué información recibe cada procedimiento a través de sus parámetros?
50. ¿Qué ocurriría si un parámetro recibiera un valor inesperado?
51. ¿Cómo se ejecuta el procedimiento y cómo comprobamos su resultado?
52. ¿Qué significa que un procedimiento pueda reutilizarse desde otros programas PL/SQL?
53. ¿Qué ventajas ofrece encapsular una operación de negocio en un procedimiento?
54. ¿Qué desventajas o limitaciones podría tener nuestra implementación?
55. ¿Cómo identificaríamos si un procedimiento modifica más registros de los que debería?
56. ¿Qué ocurriría si la operación se ejecutara dos veces?
57. ¿Cómo podríamos adaptar el procedimiento si cambiara una regla de negocio?
58. ¿Qué decisión de diseño relacionada con los procedimientos fue la más importante en nuestro proyecto?

6. Funciones almacenadas

59. ¿Qué necesidad del proyecto resuelve cada función?
60. ¿Qué diferencia existe entre una función almacenada y un procedimiento almacenado?
61. ¿Qué valor devuelve cada función y qué representa ese resultado?
62. ¿Por qué una función podría resultar conveniente para calcular un indicador o consultar información?
63. ¿Qué diferencia existe entre una función con parámetros y una sin parámetros?
64. ¿Cómo podemos utilizar una función desde una sentencia SQL?
65. ¿Qué restricciones debemos considerar al invocar una función desde SQL?
66. ¿Qué sucedería si la función recibiera valores nulos o datos inesperados?
67. ¿Cómo comprobaríamos que el valor devuelto es correcto?
68. ¿Cuándo sería preferible utilizar una consulta SQL en lugar de una función almacenada?
69. ¿Qué limitaciones tendría nuestra función si necesitáramos procesar grandes volúmenes de información?
70. ¿Qué cambios realizaríamos si el negocio necesitara calcular un indicador adicional?

7. Packages: organización y encapsulamiento

71. ¿Qué problema de organización del código resuelve el package que desarrollamos?
72. ¿Qué diferencia existe entre la especificación y el cuerpo de un package?
73. ¿Qué elementos de nuestro package son públicos y cuáles son privados?
74. ¿Por qué decidimos exponer algunos procedimientos o funciones y mantener otros privados?
75. ¿Qué significa que un elemento sea accesible desde fuera del package?
76. ¿Qué ventajas ofrece agrupar procedimientos y funciones relacionados?
77. ¿Cómo se relaciona nuestro package con los demás objetos de la base de datos?
78. ¿Qué ocurriría si necesitáramos modificar la implementación interna de una función privada?
79. ¿Qué cambios podrían afectar a los programas que utilizan los elementos públicos del package?
80. ¿Qué ventajas ofrece mantener una interfaz pública estable?
81. ¿Qué errores podrían aparecer si la especificación y el cuerpo del package no fueran coherentes?
82. ¿Qué dificultades encontramos al solicitar a la IA que generara o modificara el package?
83. ¿Cómo comprobamos que los objetos contenidos en el package funcionan correctamente?
84. ¿Qué elementos no sería conveniente exponer públicamente en nuestra solución y por qué?

8. Triggers: automatización y control de operaciones

85. ¿Qué regla de negocio o necesidad de control resuelve cada trigger?
86. ¿Qué diferencia existe entre un trigger a nivel de fila y uno a nivel de sentencia?
87. ¿Qué diferencia existe entre ejecutar un trigger antes o después de una operación?
88. ¿Qué eventos pueden activar nuestros triggers?
89. ¿Qué información puede consultar un trigger a nivel de fila mediante los valores `:OLD` y `:NEW`?
90. ¿Cómo comprobamos que el trigger se activa cuando corresponde?
91. ¿Qué sucede si una operación afecta a varias filas?
92. ¿Cómo verificaríamos cuántas veces se ejecuta un trigger durante una operación que modifica múltiples registros?
93. ¿Qué riesgos existen al utilizar triggers para implementar reglas de negocio?
94. ¿Por qué un trigger puede dificultar la comprensión del comportamiento de una tabla?
95. ¿Qué ocurriría si una operación activara indirectamente otros triggers?
96. ¿Qué es una tabla mutante y en qué circunstancias puede aparecer ese problema?

97. ¿Cómo podríamos evitar una implementación que produzca errores de tabla mutante?
98. ¿Qué lógica de nuestro proyecto podría haberse implementado mediante procedimientos en lugar de triggers?
99. ¿Por qué elegimos utilizar un trigger y no otra alternativa?
100. ¿Qué pruebas realizamos para comprobar que el trigger no se activa en situaciones incorrectas?

9. Integración de los objetos PL/SQL

101. ¿Cómo se relacionan los procedimientos, funciones, packages y triggers de nuestro proyecto?
102. ¿Qué componente cumple la función principal dentro de la solución?
103. ¿Qué objetos dependen de otros objetos?
104. ¿En qué orden deben crearse o compilarse los objetos?
105. ¿Qué ocurriría si modificáramos una función utilizada por un package?
106. ¿Cómo identificaríamos las dependencias entre los objetos desarrollados?
107. ¿Qué estrategia utilizaríamos para probar los componentes de manera individual?
108. ¿Cómo verificaríamos que todos funcionan correctamente cuando se integran?
109. ¿Qué errores pueden aparecer cuando los componentes funcionan por separado, pero no en conjunto?
110. ¿Qué decisiones tomamos para evitar duplicación de lógica?
111. ¿Cómo se comportaría nuestra solución ante una operación inválida?
112. ¿Qué modificaciones serían necesarias si aumentara considerablemente el volumen de datos?
113. ¿Qué parte de la solución sería más difícil de mantener y por qué?
114. ¿Qué cambiaríamos si tuviéramos que diseñar nuevamente el proyecto?

10. Errores, depuración y evaluación crítica de la IA

Los errores forman parte del proceso de desarrollo. Lo importante es comprenderlos y demostrar cómo fueron abordados.

115. ¿Cuál fue el error más relevante que encontramos durante el desarrollo?
116. ¿Qué mensaje de error obtuvimos y qué significaba?
117. ¿Cómo identificamos la causa del error?
118. ¿Qué información le proporcionamos a la IA para solicitar ayuda?
119. ¿La primera respuesta de la IA resolvió el problema? ¿Cómo lo comprobamos?
120. ¿Qué hicimos cuando una solución sugerida por la IA no funcionó?

121. ¿Cómo distinguimos entre un error de sintaxis, un problema de lógica y un problema de diseño?
122. ¿Qué error nos permitió comprender mejor un concepto de PL/SQL?
123. ¿Qué respuesta de la IA consideramos incorrecta, incompleta o poco apropiada?
124. ¿Qué evidencia nos permitió cuestionarla?
125. ¿Qué riesgos tendría aceptar automáticamente una solución sin probarla?
126. ¿Cómo documentamos los cambios realizados para corregir un problema?
127. ¿Qué aprendimos de un error que no habríamos identificado si la primera versión hubiera funcionado?
128. ¿Qué estrategia de depuración utilizaríamos si volviéramos a encontrar un problema similar?

11. Diseño de pruebas y comprobación de resultados

129. ¿Qué comportamiento esperamos observar en cada objeto?
130. ¿Qué datos de prueba seleccionamos y por qué?
131. ¿Qué casos normales probamos?
132. ¿Qué casos límite o excepcionales consideramos?
133. ¿Cómo probamos el comportamiento ante datos nulos, inexistentes o inesperados cuando corresponda?
134. ¿Qué resultado esperábamos antes de ejecutar cada prueba?
135. ¿Qué resultado obtuvimos realmente?
136. ¿Cómo explicamos las diferencias entre el resultado esperado y el obtenido?
137. ¿Cómo comprobamos que un trigger no modifica datos cuando no debería hacerlo?
138. ¿Cómo verificamos que los procedimientos no alteran registros ajenos a la operación solicitada?
139. ¿Qué pruebas repetiríamos después de modificar el código?
140. ¿Qué evidencia mostraríamos a una persona que quisiera comprobar independientemente nuestros resultados?
141. ¿Qué escenarios todavía no hemos probado?
142. ¿Qué limitaciones tiene nuestra estrategia de pruebas?

12. Reflexión sobre el aprendizaje y el uso de IA

Esta sección invita a reflexionar sobre el proceso, no a presentar una experiencia idealizada.

143. ¿Qué sabíamos sobre PL/SQL antes de comenzar?
144. ¿Qué conceptos comprendemos mejor después de desarrollar el proyecto?

145. ¿Qué aprendimos sobre procedimientos, funciones, packages o triggers que antes no entendíamos suficientemente?
146. ¿Qué tarea nos resultó más difícil y por qué?
147. ¿En qué momento la IA contribuyó más a nuestro aprendizaje?
148. ¿En qué momento la IA dificultó nuestra comprensión o nos llevó por un camino poco adecuado?
149. ¿Qué descubrimos al probar el código que no aparecía claramente en la explicación de la IA?
150. ¿Qué hicimos para comprobar que realmente habíamos comprendido una respuesta?
151. ¿Qué decisiones tomamos como equipo sin delegarlas completamente en la IA?
152. ¿Qué haríamos de manera diferente si comenzáramos el proyecto nuevamente?
153. ¿Qué parte del código nos sentimos capaces de explicar y modificar sin asistencia inmediata de la IA?
154. ¿En qué aspectos todavía necesitaríamos ayuda?
155. ¿Qué aprendimos sobre la formulación de preguntas técnicas?
156. ¿Qué diferencia notamos entre obtener una respuesta y comprender una solución?
157. ¿Qué prácticas de uso de IA mantendríamos en un futuro trabajo profesional?
158. ¿Qué limitaciones debemos reconocer cuando utilizamos IA para desarrollar software?
159. ¿Qué responsabilidad tenemos respecto del código que entregamos, aunque haya sido generado por una herramienta?
160. ¿Qué evidencia concreta demuestra nuestro aprendizaje durante el proyecto?

13. Preguntas de preparación para la defensa individual

Durante la defensa, el docente podrá solicitar explicaciones, demostraciones, justificaciones o análisis de situaciones hipotéticas. No bastará con leer el informe ni con reconocer visualmente el código.

161. Explique el problema de negocio y la solución desarrollada por su equipo.
162. Seleccione un procedimiento y explique su funcionamiento paso a paso.
163. ¿Por qué ese procedimiento es necesario en el proyecto?
164. Explique una función almacenada e indique qué valor devuelve.
165. ¿Cómo se utiliza esa función desde SQL?
166. Explique la organización de su package y distinga sus elementos públicos y privados.
167. ¿Por qué se tomó esa decisión de encapsulamiento?

168. Seleccione un trigger y explique qué evento lo activa.
169. ¿Qué diferencia habría si ese trigger fuera a nivel de sentencia en lugar de fila, o viceversa?
170. Muestre una prueba que permita comprobar el funcionamiento de uno de sus objetos.
171. ¿Qué resultado esperaba y cómo interpreta el resultado obtenido?
172. Seleccione un error que haya encontrado y explique cómo lo resolvió.
173. ¿Qué respuesta de la IA tuvo que revisar, corregir o adaptar?
174. ¿Cómo supo que la solución propuesta por la IA era correcta?
175. Si tuviera que modificar una condición del problema, ¿qué parte del código cambiaría?
176. Si una función devolviera un resultado incorrecto, ¿cómo investigaría la causa?
177. Si un procedimiento modificara registros que no debería modificar, ¿qué revisaría?
178. Si un trigger se ejecutara en una situación inesperada, ¿cómo identificaría el problema?
179. Si el proyecto tuviera que procesar muchos más registros, ¿qué aspectos evaluaría?
180. ¿Qué alternativa técnica podría resolver el mismo problema y por qué eligió la solución actual?
181. ¿Qué parte de su solución considera más sólida y qué parte necesita mejorar?
182. ¿Qué concepto comprendió mejor gracias a una dificultad concreta durante el desarrollo?
183. ¿Qué cambiaría si tuviera que implementar nuevamente el proyecto?
184. ¿Qué responsabilidad asume sobre el código presentado, independientemente de quién o qué herramienta lo haya generado?

14. Preparación personal antes de exponer

Antes de la defensa, cada integrante debería poder:

- Explicar el propósito de los principales objetos desarrollados.
- Justificar las decisiones técnicas más importantes.
- Diferenciar procedimientos, funciones, packages y triggers.
- Identificar las relaciones y dependencias entre los objetos.
- Mostrar pruebas y explicar los resultados obtenidos.
- Describir al menos una dificultad técnica y cómo fue abordada.
- Reconocer limitaciones o aspectos mejorables de la solución.
- Explicar de qué manera utilizó la IA y cómo verificó sus respuestas.
- Responder preguntas sin depender exclusivamente del texto del informe.
- Analizar una modificación o situación hipotética relacionada con el proyecto.

Cierre: El propósito de la evaluación es demostrar que pueden trabajar con IA de manera responsable y técnicamente fundamentada. Una solución generada con ayuda de IA puede ser excelente; lo que deben demostrar individualmente es que comprenden su funcionamiento, pueden evaluar su calidad y son capaces de tomar decisiones informadas sobre ella.